

P4 Programming for Tofino

Shivam Agarwal

f20212909@goa.bits-pilani.ac.in

[Introduction](#)

[General Notes and Resources](#)

[Writing P4 code for hardware](#)

[1\) General](#)

[2\) Parser](#)

[3\) Processing](#)

[Running P4 code on Physical Switch](#)

[Directory Structure](#)

[1. Precursor](#)

[2. Compile P4 Program](#)

[3. Build P4 Program](#)

[4. Load Program](#)

[5. Run Program](#)

[Inside BFShell](#)

[Port Configurations - UCLI](#)

[Adding Table Entries BFRT](#)

[Tests](#)

[1\) Between PC and Laptop](#)

[2\) Between PC1 and PC2](#)

[3\) Basic Port to Port Forwarding](#)

[4\) iPerf tests](#)

Introduction

This project has been about setting up the Edgecore Wedge100BF-32x [DCS800 – Edgecore Networks \(edge-core.com\)](#) programmable switch and running P4 programs on it.

There is a repo associated with this report. This contains all the raw experiment data pcaps, images, etc. [axlrpro/p4-experiments: Documentation for various P4 experiments performed in lab at BITS Goa \(github.com\)](#)

General Notes and Resources

There are almost no resources for beginner learners. All information is scattered, and basic Google searches do not usually help because they take you down a spiral of information often not required.

That being said, the resources I found most helpful.

- For basic network knowledge
 - [Welcome to The TCP/IP Guide! \(tcpipguide.com\)](#) - A detailed guide for network protocols. Use this if you want to understand every aspect of a topic.
 - Google searches - Basic network questions can be answered by Google searches.
- P4 switches

P4 switches do not work the usual way. To get an overview of how packets move in the switch, read this first

 - [P4~16~ Portable Switch Architecture \(PSA\)](#)
 - [Tofino-Native-Arch](#) - Intel's Tofino architecture for PSA switches. Our switch, Edgecore Wedge100BF-32X, follows the Tofino1 architecture.
- Writing P4 code
 - [p4 Tutorials](#) - The best place to start learning P4 code. Start with [basic_forwarding](#). This is for simulation.
 - [tna-arch](#) - Defines the structs and definitions of parsers for the TNA architecture. Write code in the TNA architecture to run on the physical switch.
- P4 language definition

Have not required much of this.

- [P4~16~ Language Specification](#)
- A good starting point. Worth going through the whole doc - [\(PDF\) Bachelor Thesis: "Building a training platform for the programming language P4" \(researchgate.net\)](#)
- If you're stuck on a P4 programming-specific question, try the [P4 forum](#)
- Try the Intel Tofino Forum if you are stuck on a Tofino Programming-specific question.

Writing P4 code for hardware

There are two ways we can test our P4 code

- 1) Simulation in mininet
- 2) Physical Edgecore switches

The simulation switches follow BehavioralModel architecture, [bmv2](#), whereas the physical switch follows the Tofino Native Architecture, [tna](#). Due to this, there are multiple differences in how code is written for both. I have covered a few of the basic differences here. This list is not exhaustive, and you can look at the definitions in [Open-Tofino/share/p4c/p4include at master · barefootnetworks/Open-Tofino \(github.com\)](#).

The definitions are split up into the following files:

- [tna.p4](#) - top-level file to be included in P4 code
- [tofino1_specs.p4](#) - Target-device-specific types, constants, and macros
- [tofino1_arch.p4](#) - parser, control, and packages definitions
- [tofino1_base.p4](#) - portable constants, headers, and externs, etc

1) General

BMv2

There are six control blocks. The order of these is fixed, and all must be present for the program to be valid.

- 1) Parser
- 2) Checksum Verification

- 3) Ingress Processing
- 4) Egress Processing
- 5) Checksum Verification
- 6) Deparser

P4 code for BMv2 must include the header

```
#include <v1model.p4>
```

In code:

```
V1Switch(  
  MyParser(),  
  MyVerifyChecksum(),  
  MyIngress(),  
  MyEgress(),  
  MyComputeChecksum(),  
  MyDeparser()  
) main;
```

TNA

There are six control blocks. The order is fixed, and all must be present for the program to be valid.

- 1) IngressParser
- 2) Ingress
- 3) IngressDeparser
- 4) EgressParser
- 5) Egress
- 6) EgressDeparser

P4 code for TNA architecture must include

```
#include <tna.p4>
```

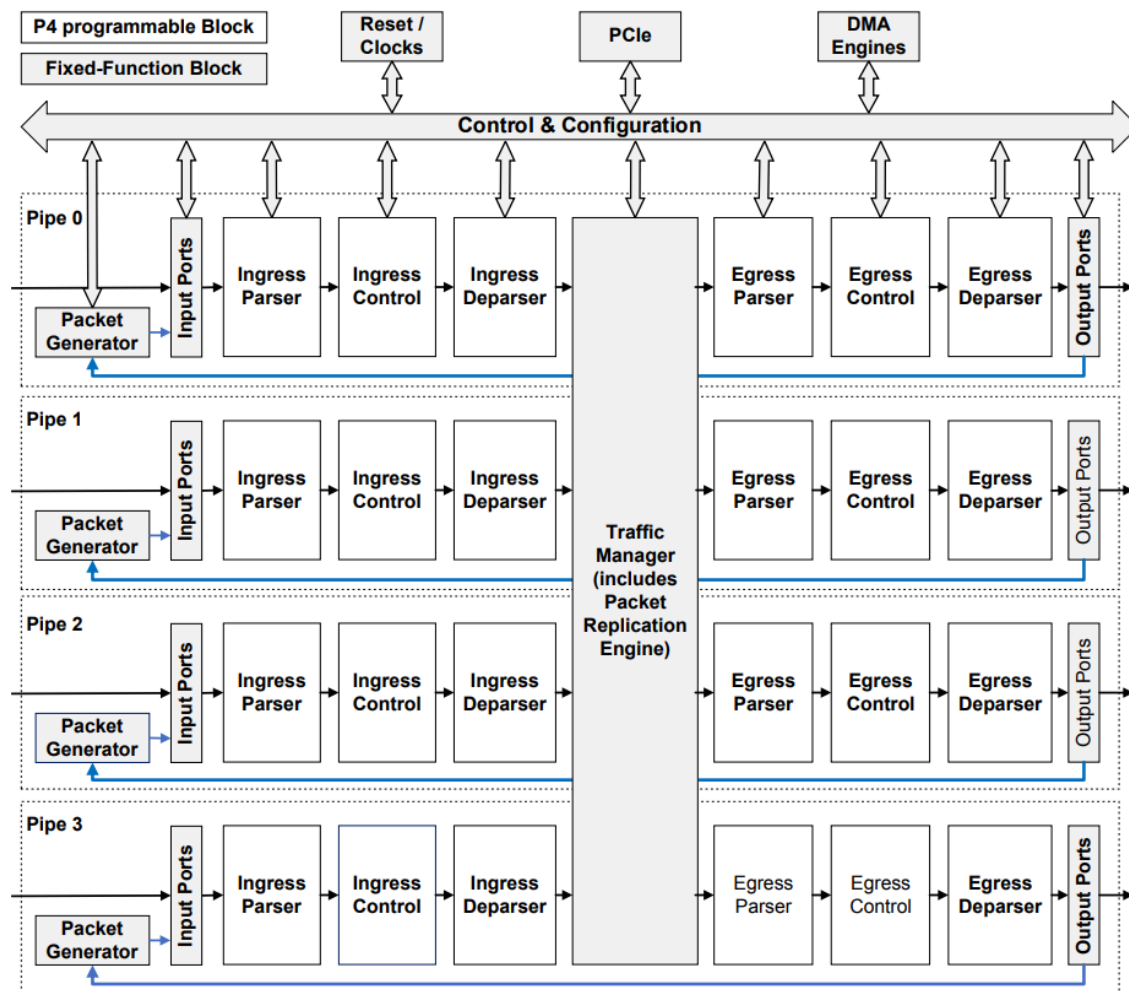
In code:

```
Pipeline(  
  IngressParser(),  
  Ingress(),  
  IngressDeparser(),  
  EgressParser(),  
  Egress(),  
  EgressDeparser()  
)
```

```
) pipe;
```

```
Switch(pipe) main;
```

The complete TNA architecture. TM stands for Traffic Manager, and PRE stands for Packet Replication Engine. TM stores and queues the packets in a buffer until they are sent and implements the commands from the code, e.g., setting the egress port. PRE is used for packet duplication (multicast). They obtain their data from the Intrinsic Metadata and table entries.



TNA Architecture Diagram

2) Parser

BMv2

```

parser MyParser(packet_in packet,
                out headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        transition accept;
    }
}

```

TNA

```

parser MyIngressParser(packet_in packet,
                       out headers hdr,
                       out metadata meta,
                       out ingress_intrinsic_metadata_t ig_intr_md
                       ) {

    state start {
        packet.extract(ig_intr_md);    // REQUIRED
        transition accept;
    }
}

```

3) Processing

The parser is followed by processing blocks, where you can change packet headers and set egress_ports, etc. Decisions are made based on the Match-Action principle. You can define the Match key and declare the action in the code. Actions can be hardcoded (e.g., always forward any packet ingress on port 1 to port 2) or dynamic (e.g., L2 switching, where the switch learns which MAC address is associated with which port). P4 code can only declare an action and define hardcoded actions. For dynamic actions, the control plane must be involved.

Example 1: A sample code without any match-action logic that forwards traffic between two ports. I.e., incoming traffic on port 1 forwarded to port 2 and incoming packet on port 2 forwarded to port 1.

BMv2

```
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
  apply {
    if (standard_metadata.ingress_port == 1) {
      standard_metadata.egress_spec = 2;
    }
    else if (standard_metadata.ingress_port == 2) {
      standard_metadata.egress_spec = 1;
    }
  }
}
```

- `standard_metadata.ingress_port` - tells you the physical port from which the packet was received
- `standard_metadata.egress_spec` - helps you set the egress port, the port you want to route the packet to

TNA

```
control Ingress(inout headers hdr,
                inout metadata meta,
                in ingress_intrinsic_metadata_t ig_intr_md,
                in ingress_intrinsic_metadata_from_parser_t ig_prsr_md,
                inout ingress_intrinsic_metadata_for_deparser_t ig_dprsr_md,
                inout ingress_intrinsic_metadata_for_tm_t ig_tm_md)
{
  apply{
    if(ig_intr_md.ingress_port == 1){
      ig_tm_md.ucast_egress_port = 2;
    }
    else if(ig_init_md.ingress_port == 2){
      ig_tm_md.ucast_egress_port = 1;
    }
  }
}
```

- `Ig_intr_md.ingress_port` - analogous to `standard_metadata.ingress_port` in bmv2
- `ig_tm_md.egress_port` - analogous to `standard_metadata.egress_spec`

Example 2: Utilizing match-action behavior. We match egress ports based on the destination IP address. Here, we need the program to maintain a table with mappings of `dst_ip` and `egress_port`.

This is what a table looks like. The following fields must be defined for a table. The table entries, however, are not defined using P4 code. This means that just writing P4 code will not forward your packets.

table_ipv4_exact	
keys	The value in the packet to match on <code>dst_addr</code>
actions	Possible actions to take <code>send</code> <code>drop</code>
default action	If a match cannot be found <code>drop</code>
size	The number of entries the table can store

BMv2

```
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
    # this makes the switch drop the packets
    action drop() {
        mark_to_drop(standard_metadata);
    }

    # to forward the packet to a port
    action send(egressSpec_t port) {
        standard_metadata.egress_spec = port;
    }

    table ipv4_exact {
        key = {
            hdr.ipv4.dstAddr: exact;
        }
    }
}
```

```

        actions = {
            send;
            drop;
        }
        size = 1024;
        default_action = drop();
    }

    apply {
        if (hdr.ipv4.isValid()) {    # check if the header is present
            ipv4_exact.apply();      # apply the table on the packet
        }
    }
}

```

TNA

```

control Ingress(
    inout my_ingress_headers_t          hdr,
    inout my_ingress_metadata_t         meta,
    in  ingress_intrinsic_metadata_t    ig_intr_md,
    in  ingress_intrinsic_metadata_from_parser_t  ig_prsr_md,
    inout ingress_intrinsic_metadata_for_deparser_t  ig_dprsr_md,
    inout ingress_intrinsic_metadata_for_tm_t    ig_tm_md)
{
    # set the egress port
    action send(PortId_t port) {
        ig_tm_md.ucast_egress_port = port;
    }

    action drop() {
        ig_dprsr_md.drop_ctl = 1;
    }

    table ipv4_exact {
        key = {
            hdr.ipv4.dst_addr : exact;
        }
    }
}

```

```

        actions = {
            send;
            drop;
        }
        default_action = drop();
        size = IPV4_HOST_TABLE_SIZE;
    }

    apply {
        if (hdr.ipv4.isValid()) {
            ipv4_exact.apply();
        }
    }
}

```

Similarly, definitions for other blocks can be seen

[Open-Tofino/share/p4c/p4include at master · barefootnetworks/Open-Tofino \(github.com\)](https://github.com/barefootnetworks/Open-Tofino/blob/master/share/p4c/p4include) for TNA.

Until now, we have only declared the tables and the possible actions. The code will not forward packets yet. We need to add the entries into the tables at runtime, as shown in the next section.

dst_addr	egress_port
10.10.1.1	21
10.10.1.2	22

Running P4 code on a Physical Switch

To run a P4 program on the switch, the program must have been written for the Tofino Native Architecture(TNA), since the switch follows TNA. See [Writing P4 code for hardware](#)

Directory Structure

We use the `developer` user for our purposes. In the `developer` user's home directory, there are two folders:

```
scripts/  
code/
```

Any P4 code you write has to be placed inside `code/` in a folder named by the program's name as "`code/<p4-program-name>/<p4-program-name>.p4`"

E.g., `code/simple_l3/simple_l3.p4` If you have P4 code written for the switch, do the following.

1. Precursor

Any time the switch is rebooted, the following has to be performed. This has to be performed to set path variables for later steps. So if the following command prints an empty line,

```
# echo $SDE
```

you need to perform the following steps

```
# cd ~/scripts  
# source set_sde_path
```

2. Compile P4 Program

```
# cd <path to your p4 code>  
# $SDE_INSTALL/bin/bf-p4c <p4-program-name>.p4
```

3. Build P4 Program

```
# cd ~/scripts/  
# source p4build_command_param <p4-program-name>  
# make  
# make install
```

4. Load Program

```
# cd ~/scripts/  
# source load_program.sh
```

5. Run Program

```
# cd $SDE  
# ./run_switchd.sh -p <p4-program-name>
```

This runs the bfshell. You can now use bfshell to access physical switch ports and add table entries.

Inside the bfshell

This is how you communicate with the switch (read/write tables, setup ports, etc.) at runtime. There are multiple tools inside bfshell.

- At any level, you can press **tab** to list all the possible options at that level.
- Use **..** to go back a level.
- To exit the bfshell, type **ctrl+**

```
bfshell>  
access      bfrt_python  debug_python  exit          help          pipemgr  
quit        run_file     ucli          version
```

Port Configurations - UCLI

To start forwarding packets using the switch, you will first have to set up and enable the physical ports on the switch. Our switch, Edgecore Wedge100BF-32x, has 32 physical QSFP28 ports. These are also called the front panel ports. Each port supports speeds of only 100Gbps and 40Gbps. This means that only cables supporting 100Gbps and

40Gbps can be used with these ports. You can connect breakout cables to the ports, separating each port into 4 different ports. We have used this configuration to set up our switch. Knowing the speed of your port and cables is important because we have to set up the ports according to their speeds in ucli.

Inside the bfshell,

```
bfshell> ucli
bf-sde> bf_pltfrm qsfp show
```

```
bfshell>
Starting UCLI from bf-shell

Cannot read termcap database;
using dumb terminal settings.
bf-sde> bf_pltfrm qsfp show
Max ports supported ==> 32
Temperature monitor enabled ==> NO
=====
Show for QSFP :
=====
```

port	id	vendor	part_num	eth cpl	ext-eth cpl	Om-len	Cu-Len	Qsfp-type
19	0x0d	OEM	CAB-Q10/4S-P1M	0x08	0x00	--0--	1 m	Copper 1 m

```
=====
Show for CMIS:
=====
```

port	module-type	vendor	part_num	Cu-Len	conn-type
port	module-type	vendor	part_num	Optics	conn-type

```
=====
bf-sde> |
```

This lists all the physical ports connected to cables that can be used to send packets. An important thing to note is the **port** column. This is the physical port number.

```
bf-sde>pm
bf-sde.pm>show
```

This shows all the ports that have been set up and are ready for transmission. To use the ports, you have to add them as follows.

```
//1- add port
port-add <port_str> <speed (1G, 10G, 25G, 40G,
40G-R2, 50G(50G/50G-R2, 50G-R2-C, 50G-R1), 100G(100G/100G-R4, 100G-R2,
100G-R1), 200G(200G/200G-R4, 200G-R2, 200G-R8), 400G(400G/400G-R8,
400G-R4))> <fec (NONE, FC, RS)>

//2- set auto negotiation
an-set <port-str> <autonegotiation (0 = on, 2 = off)>

//3- port enable
port-enb <port-str>
```

Breakout ports appear as <physical-port-number>/<breakout-number>. E.g., If port 19 is broken out into 4 ports, you can access the individual ports using 19/0, 19/1, 19/2, and 19/3. FEC stands for Forward Error Correction. Generally NONE. Autonegotiation is used when two directly connected devices can mutually decide connection parameters between them (OFF for our purposes).

E.g.,

```
//1
bf-sde.pm>port-add 19/- 10G NONE

//2
an-set 19/- 2

//3
port-enb 19/-
```

- 1) This sets all ports on physical port 19 to a speed of 10Gbps (since we are using a breakout cable of four cables with 10G for each port)
- 2) Autonegotiation is generally off
- 3) enable port

You can see the list of ports added as follows.

```
bf-sde.pm>show
```

```

bf-sde>
bf-sde> pm
bf-sde.pm> show

```

PORT	MAC	D_P	P/PT	SPEED	FEC	AN	KR	RDY	ADM	OPR	LPBK	FRAMES RX	FRAMES TX	E
19/0	5/0	20	1/20	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	
19/1	5/1	21	1/21	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	
19/2	5/2	22	1/22	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	
19/3	5/3	23	1/23	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	

```

bf-sde.pm> show

```

PORT	MAC	D_P	P/PT	SPEED	FEC	AN	KR	RDY	ADM	OPR	LPBK	FRAMES RX	FRAMES TX	E
19/0	5/0	20	1/20	10G	NONE	Ds	Au	YES	ENB	UP	NONE	27	0	
19/1	5/1	21	1/21	10G	NONE	Ds	Au	YES	ENB	UP	NONE	0	0	
19/2	5/2	22	1/22	10G	NONE	Ds	Au	YES	ENB	UP	NONE	36	0	
19/3	5/3	23	1/23	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	

```

bf-sde.pm> ..
bf-sde>

```

- **D_P** - Virtual port numbers (used later for setting up tables)
- **ADM** - **DOWN** if the port is disabled. **UP** if enabled.
- **OPR** - Operability. **UP** - If the port is enabled, connected to a cable, and can transmit packets. **DOWN** - otherwise
- **FRAMES_RX** - number of packets received at port
- **FRAMES_TX** - number of packets transmitted from port

To exit, UCLI and go back to **bfshell** type **exit**

```

bf-sde>exit
bfshell>

```

Adding Table Entries bfrt

BFRT stands for BarefootRuntime. This is the control plane analogous to P4Runtime. We can add table entries using **bfrt**. Auto-completion can be done using **tab**. Use **dump** to list table entries and **info** to see table size.

The tables are accessed as follows.

```

bfrt.<p4-program-name>.pipe.<control-block-name>.<table>

```

The tables are edited as follows.

```
// add a new entry  
add_with_<action-name>(<key>, <parameter>)  
  
//edit entry  
mod_with_<action-name>(<key>, <parameter>)  
  
//set default action as <action-name>  
set_default_with_<action-name>(<key>, <parameter>)  
  
//delete an entry for a certain key  
delete(<key>)
```

Navigating to the table

```
bfshell>
access      bfrt_python  debug_python  exit          help          pipemgr
quit        run_file    ucli          version
bfshell> bfrt_python
cwd : /home/developer/bf-sde-9.13.3

We've found 1 p4 programs for device 0:
simple_l3
Creating tree for dev 0 and program simple_l3

Devices found : [0]
bPython 3.10.9 (main, Jul 10 2024, 07:31:24) [GCC 11.4.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 7.31.1 -- An enhanced Interactive Python. Type '?' for help.

bfrt_python> bfrt
-----> bfrt()
Available symbols:
dump          - Command
info          - Command
mirror        - Node
port          - Node
pre           - Node
simple_l3      - Node
tfl           - Node

bfrt> simple_l3
-----> simple_l3()
Available symbols:
dump          - Command
info          - Command
pipe          - Node
tbl_dbg_counter - Table

bfrt.simple_l3> pipe
-----> pipe()
Available symbols:
Ingress       - Node
IngressParser - Node
dump          - Command
info          - Command
snapshot      - Node
```

Navigating to table

```
bfrt.simple_l3.pipe> Ingress
-----> Ingress()
Available symbols:
dump          - Command
info          - Command
ipv4_host     - Table
ipv4_lpm      - Table

bfrt.simple_l3.pipe.Ingress> ipv4_host.dump
-----> ipv4_host.dump()
----- ipv4_host Dump Start -----
Default Entry:
Entry data (action : NoAction):

Table pipe.Ingress.ipv4_host has no entries.
----- ipv4_host Dump End -----
```

Using **tab** to get hints about available actions in table **ipv4_host**.

```
bfrt.simple_l3.pipe.Ingress> ipv4_host.add_with_
ipv4_host.add_with_drop
ipv4_host.add_with_send
```

Note: bfrt recognizes virtual port numbers and not physical port numbers. So we have to enter the port number corresponding to **D_P** column in **ucli**.

```
bf-sde.pm> show
```

PORT	MAC	D_P	P/PT	SPEED	FEC	AN	KR	RDY	ADM	OPR	LPBK	FRAMES RX	FRAMES TX	E
19/0	5/0	20	1/20	10G	NONE	Ds	Au	YES	ENB	UP	NONE	27	0	0
19/1	5/1	21	1/21	10G	NONE	Ds	Au	YES	ENB	UP	NONE	0	0	0
19/2	5/2	22	1/22	10G	NONE	Ds	Au	YES	ENB	UP	NONE	36	0	0
19/3	5/3	23	1/23	10G	NONE	Ds	Au	YES	ENB	DWN	NONE	0	0	0

In this example, the physical port is 19, but the virtual ports are 21 and 22. Since this is a breakout port, there are 4 virtual ports associated with one physical port.

Tests

Multiple tests have been run on the system. The terminology and devices used are as follows:

- ECS4150-28T - [ECS4150-28T – Edgecore Networks \(edge-core.com\)](https://www.edge-core.com/products/eecs4150-28t/). Also called the **L2 switch**. It has multiple RJ45 ports and 4 SFP+ ports.
- Wedge100BF-32X - [DCS800 – Edgecore Networks \(edge-core.com\)](https://www.edge-core.com/products/wedge100bf-32x/). The programmable switch. P4 codes are run on this switch. It has 32 QSFP28 ports.
- Ubuntu PCs - 2 PCs running Ubuntu having SFP+ ports.
- Laptop - a laptop running Windows. Has one RJ45 port.
- Campus Net - The campus-wide network through the wired ethernet.
- 40G-10G breakout cable - This cable helps connect one 40Gbps port to four 10Gbps ports.
- RJ45-RJ45 ethernet cable - Standard ethernet cable connecting two RJ45 ports.

Raw data and images are stored here: [axlrpro/p4-experiments: Documentation for various P4 experiments performed in lab at BITS Goa \(github.com\)](https://github.com/axlrpro/p4-experiments)

1) Between PC and Laptop

Summary

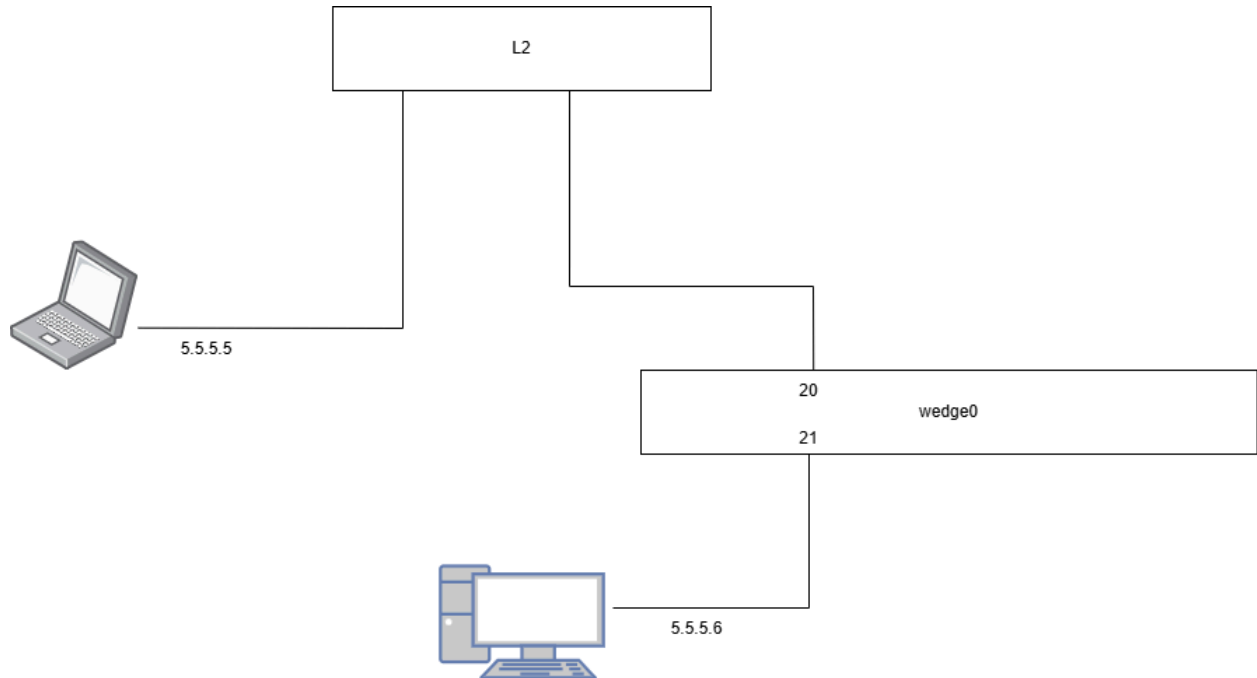
Test	Tables Configured	Source	Dest	Packet Details			Observation
				src_ip	dst_ip	interface	
1	No	PC	Laptop	5.5.5.6	5.5.5.5	enp1s0	Packets not received at the dest
2	Yes	PC	Laptop	5.5.5.6	5.5.5.5	enp1s0	Packets received at the dest
3	Yes	Laptop	PC	5.5.5.5	5.5.5.6	"Realtek..."	Packets received at the dest

ipv4_host	
key: dest_ip	action: send(port)
5.5.5.5	20
5.5.5.6	21

Table entries

Physical Setup

- The PC is connected to the Wedge switch using a 40G-10G breakout cable.
- The Wedge switch is connected to the L2 Switch using a 40G-10G breakout cable.
- A Laptop is connected to the L2 switch using RJ45-RJ45 ethernet cable.



Topology for test

Codes

- The wedge switch has been set up with `simple_l3.p4` code. This code enables IP-based port forwarding. It uses destination IP as the key and sets egress_ports accordingly
- Packets are sent using `send_pkt.py` script

```
from scapy.all import *
import sys

args = sys.argv[1:]
if(len(args)!=5):
    print("usage:  send_pkt.py  <src_ip>  <dst_ip>  <msg>  <interface> <repetitions>")
    exit(1)

src_ip = args[0]
dst_ip = args[1]
msg = args[2]
iface = args[3]
repetitions = int(args[4])

for i in range(repetitions):
```

```
p = Ether()/IP(dst=dst_ip, src=src_ip)/UDP()/msg
sendp(p, iface=iface)
```

- The interface can be found using the `list_interfaces.py` script

```
from scapy.all import *
from scapy.interfaces import *

show_interfaces()
print(get_if_list())
print(conf.ifaces)
```

- Packets are captured using Wireshark.

a) Sending packets from PC to Laptop - with empty switch tables

The tables are empty, and the default action is `drop`.

Packets are sent from the PC using `send_pkt.py` as follows:

```
# python3 send_pkt.py 5.5.5.6 5.5.5.5 Hi enp1s0 5
```

Observation: No packets are obtained on the laptop. It is as expected since there are no entries in the tables.

b) Sending packets from PC to Laptop - tables filled

The tables have been filled as such using the `bfrt_python` runtime using the `add_with_send` command.

ipv4_host	
key: dest_ip	action: send(port)
5.5.5.5	20
5.5.5.6	21

```

bfrt.simple_l3.pipe.Ingress> dump
-----> dump()
----- ipv4_lpm Dump Start -----
Default Entry:
Entry data (action : Ingress.send):
    port                                : 0x42

Table pipe.Ingress.ipv4_lpm has no entries.
----- ipv4_lpm Dump End -----

----- ipv4_host Dump Start -----
Default Entry:
Entry data (action : NoAction):

Entry 0:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050505
Entry data (action : Ingress.send):
    port                                : 0x14

Entry 1:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050506
Entry data (action : Ingress.send):
    port                                : 0x15

----- ipv4_host Dump End -----

```

Packets are sent from the PC using `send_pkt.py` as follows:

```

PC# python3 send_pkt.py 5.5.5.6 5.5.5.5 Hi enp1s0 5
PC# python3 send_pkt.py 5.5.5.6 5.5.5.5 Hi enp1s0 9

```

Observation: Packets are obtained at the Laptop. (*pcaps in repo*)

145	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
146	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
147	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
148	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
149	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
150	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002
151	2024-10-0...	EdgecoreAmer_...	LLDP_Multicast	LLDP	226	MA/90:2d:77:0c:4d:80 MA/90:2d:77:0c:4d:89 120 SysN= SysD=ECS4150-28T
152	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002
153	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002
154	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002
155	2024-10-0...	EdgecoreAmer_...	LLDP_Multicast	LLDP	226	MA/90:2d:77:0c:4d:80 MA/90:2d:77:0c:4d:89 120 SysN= SysD=ECS4150-28T
156	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
157	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
158	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
159	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
160	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
161	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
162	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
163	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
164	2024-10-0...	5.5.5.6	5.5.5.5	DNS	60	[Malformed Packet]
165	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002
166	2024-10-0...	EdgecoreAmer_...	AcctonTechno_0...	LLC	60	U, func=UI; SNAP, OUI 0x0012CF (Accton Technology Corp), PID 0x0002

From the Laptop

c) Sending packets from Laptop to PC - tabled filled

As a redundancy check, I have also sent packets from the Laptop to the switch.

Tables in the switch have been configured as in the previous example **(b)**

ipv4_host	
key: dest_ip	action: send(port)
5.5.5.5	20
5.5.5.6	21

```

bfrt.simple_l3.pipe.Ingress> dump
-----> dump()
----- ipv4_lpm Dump Start -----
Default Entry:
Entry data (action : Ingress.send):
    port                                : 0x42

Table pipe.Ingress.ipv4_lpm has no entries.
----- ipv4_lpm Dump End -----

----- ipv4_host Dump Start -----
Default Entry:
Entry data (action : NoAction):

Entry 0:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050505
Entry data (action : Ingress.send):
    port                                : 0x14

Entry 1:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050506
Entry data (action : Ingress.send):
    port                                : 0x15

----- ipv4_host Dump End -----

```

Packets are sent from Laptop using `send_pkt.py` as follows:

```

Laptop# python .\send_pkt.py 5.5.5.5 5.5.5.6 "Hey PC" "Realtek
PCIe GbE Family Controller" 3
Laptop# python .\send_pkt.py 5.5.5.5 5.5.5.6 "Hey PC" "Realtek
PCIe GbE Family Controller" 5

```

The interface was identified using `list_interfaces.py` script.

Source	Index	Name
libpcap	1	Software Loopback Interface 1
libpcap	10	VirtualBox Host-Only Ethernet Adapter
libpcap	11	Microsoft Wi-Fi Direct Virtual Adapter #2
libpcap	13	Intel(R) Wi-Fi 6 AX201 160MHz
libpcap	19	Microsoft Wi-Fi Direct Virtual Adapter
libpcap	20	WAN Miniport (Network Monitor)
libpcap	21	VMware Virtual Ethernet Adapter for VMnet8
libpcap	25	WAN Miniport (IP)
libpcap	26	Realtek PCIe GbE Family Controller
libpcap	8	WAN Miniport (IPv6)
libpcap	9	VMware Virtual Ethernet Adapter for VMnet1

Observation: Packets were obtained at the PC (*pcaps in repo*)

No.	Time	Source	Destination	Protocol	Length	Info
1	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
2	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
3	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
4	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
5	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
6	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
7	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]
8	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (15) 0x4865[Malformed Packet]

From the PC

2) Between PC1 and PC2

Summary

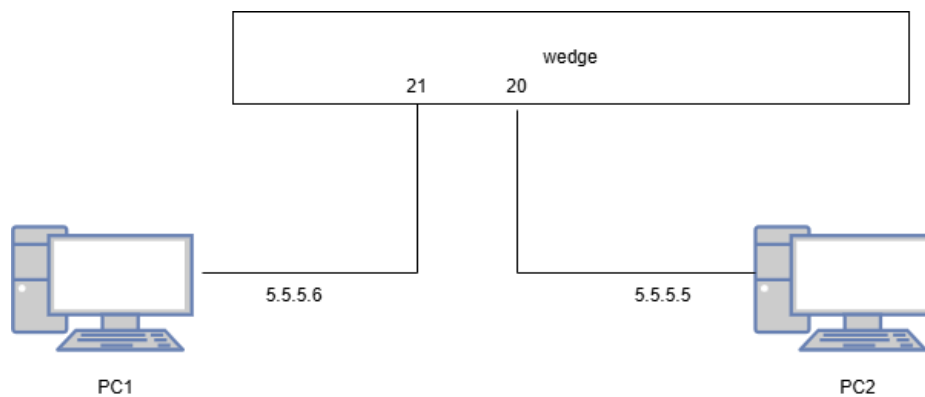
Test	Tables Configured	Source	Dest	Packet Details			Observation
				src_ip	dst_ip	interface	
1	Yes	PC2	PC1	5.5.5.5	5.5.5.6	enp1s0	Packets received at dest
2	Yes	PC1	PC2	5.5.5.6	5.5.5.5	enp3s0	Packets received at dest

ipv4_host	
key: dest_ip	action: send(port)
5.5.5.5	20
5.5.5.6	21

Table entries

Physical Setup

- PC1 is connected to Wedge at port 21 using a breakout cable
- PC2 is connected to Wedge at port 20 using a breakout cable



Topology for test

Code

- The wedge switch has been set up with `simple_l3.p4` code. This code enables IP-based port forwarding. It uses destination IP as the key and sets egress_ports accordingly.
- Packets are sent using `send_pkt.py` script

- Packets are captured using Wireshark
- Tables have been filled as follows.

ipv4_host	
key: dest_ip	action: send(port)
5.5.5.5	20
5.5.5.6	21

Table entries

```

bfrt.simple_l3.pipe.Ingress> dump
-----> dump()
----- ipv4_lpm Dump Start -----
Default Entry:
Entry data (action : Ingress.send):
    port                                : 0x42

Table pipe.Ingress.ipv4_lpm has no entries.
----- ipv4_lpm Dump End -----

----- ipv4_host Dump Start -----
Default Entry:
Entry data (action : NoAction):

Entry 0:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050505
Entry data (action : Ingress.send):
    port                                : 0x14

Entry 1:
Entry key:
    hdr.ipv4.dst_addr                  : 0x05050506
Entry data (action : Ingress.send):
    port                                : 0x15

----- ipv4_host Dump End -----

```

a) **PC2 to PC1**

Packets are sent from PC2 as follows.

```
PC2# python3 send_pkt.py 5.5.5.6 5.5.5.5 Hi_pc1 enp1s0 5
```

Observation: Packets are received at PC2 (*pcaps in repo*)

No.	Time	Source	Destination	Protocol	Length	Info
1	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
2	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
3	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
4	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
5	2024-10-0...	5.5.5.5	5.5.5.6	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]

pcaps for PC2

b) PC1 to PC2

Packets are sent from PC1 as follows.

```
PC1# python3 send_pkt.py 5.5.5.5 5.5.5.6 Hi_pc2 enp3s0 9
```

Observation: Packets are received at PC1 (*pcaps in repo*)

No.	Time	Source	Destination	Protocol	Length	Info
1	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
2	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
3	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
4	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
5	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
6	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
7	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
8	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]
9	2024-10-01 22...	5.5.5.6	5.5.5.5	DNS	60	Unknown operation (11) 0x4869[Malformed Packet]

pcaps for PC1

3) Basic Port-to-Port Forwarding

Summary

Here, I have implemented a basic forwarding between two ports. This would allow unconditional forwarding between the two ports without any logic, enabling us to connect a device to the campus network using a wedge switch.

Code

- A wedge switch has been installed with `port-forward_tna.p4`. This defines a table `port_match` with the key as the ingress port and action `forward(port)`, which sets the egress port. The table is set as follows.

port_match	
key: ig_init_md.ingress_port	action: forward(port)
20	21
21	20

Table entries

```
bfrt.p4_port_forward_tna.p.Ingress.port_match> dump
-----> dump()
----- port_match Dump Start -----
Default Entry:
Entry data (action : Ingress.drop):

Entry 0:
Entry key:
    ig_intr_md.ingress_port      : 0x14
Entry data (action : Ingress.forward):
    port                         : 0x15

Entry 1:
Entry key:
    ig_intr_md.ingress_port      : 0x15
Entry data (action : Ingress.forward):
    port                         : 0x14

----- port_match Dump End -----
```

Table entries

- `curl` is used to access the internet

Assessing without any table entries, the default table action is **drop**.

We get no response and are not able to connect to the network.

[illegible]

We can access the Internet.

4) iPerf tests

Summary

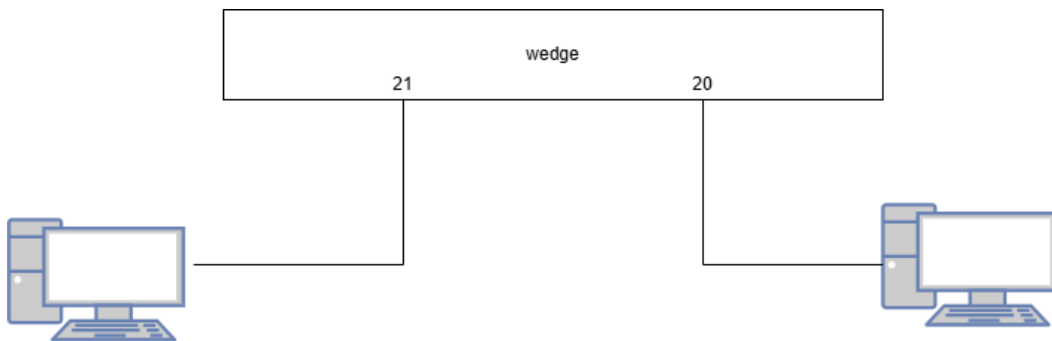
iPerf was run to test network bandwidth when connecting two devices through the Wedge switch and then through regular RJ45 Ethernet.

No.	Setup	Protocol	Bitrate	Observation
1	PC1 - Wedge - PC2	TCP	9.4 Gbits/sec	Observed bitrate close to expected (10G)
2		UDP	1.92 Gbits/sec	Observed bitrate much lower than expected (10G)
3	PC1 - L2 - Laptop	TCP	922 Mbits/sec	Observed bitrate close to expected (1G)
4		UDP	949 Mbits/sec	Observed bitrate close to expected (1G)

Procedure

a) PC to PC - through Wedge switch

- Wedge is running the `port-forward_tna.p4` program. This sends all packets ingress on port 20 to 21 and vice-versa.
- Two PCs are connected as follows.



Topology for test

- Static IP has been assigned to both PCs using `netplan`. `netplan` reads a config file and applies it to the network. The config file has to be located at `/etc/netplan/01-network-configuration-all.yaml`

```

network:
  version: 2
  renderer: NetworkManager
  ethernet:
    enp1s0:
      dhcp4: no
      addresses: [10.10.1.1/24]

```

- Changes are applied using the terminal command

```
PC# sudo netplan apply
```

- One PC has been assigned IP 10.10.1.1/24 and another 10.10.1.2/24 using the netplan

Test 1 - TCP, Wedge switch

- on PC2:

```
iperf3 -s
```

- on PC1:

```
iperf3 -c 10.10.1.2
```

Test 1 - Result

```

user@user-OptiPlex-790:~/shivan$ iperf3 -c 10.10.1.2
Connecting to host 10.10.1.2, port 5201
[ 5] local 10.10.1.1 port 48330 connected to 10.10.1.2 port 5201
[ ID] Interval           Transfer     Bitrate      Retr  Cwnd
[ 5]  0.00-1.00      sec    1.09 GBytes  9.32 Gbits/sec    10   1.26 MBytes
[ 5]  1.00-2.00      sec    1.09 GBytes  9.40 Gbits/sec     0   1.38 MBytes
[ 5]  2.00-3.00      sec    1.09 GBytes  9.40 Gbits/sec   206  1000 KBytes
[ 5]  3.00-4.00      sec    1.10 GBytes  9.42 Gbits/sec     0   1.38 MBytes
[ 5]  4.00-5.00      sec    1.09 GBytes  9.41 Gbits/sec     0   1.38 MBytes
[ 5]  5.00-6.00      sec    1.10 GBytes  9.42 Gbits/sec     0   1.40 MBytes
[ 5]  6.00-7.00      sec    1.10 GBytes  9.41 Gbits/sec     0   1.40 MBytes
[ 5]  7.00-8.00      sec    1.10 GBytes  9.42 Gbits/sec     0   1.41 MBytes
[ 5]  8.00-9.00      sec    1.09 GBytes  9.41 Gbits/sec     0   1.41 MBytes
[ 5]  9.00-10.00     sec    1.10 GBytes  9.41 Gbits/sec     0   1.42 MBytes
-----
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.00      sec    10.9 GBytes  9.40 Gbits/sec   216
[ 5]  0.00-10.04      sec    10.9 GBytes  9.36 Gbits/sec
                                     sender
                                     receiver

iperf Done.

```

Test 1 result, PC1

Test 2 - UDP, Wedge switch

- on PC2:

```
iperf3 -s
```

- on PC1: UDP, by default, limits bitrate to 1Mbps. To test at higher speeds, we need to change the max bitrate. **-b** option is used to specify max bitrate.

```
iperf3 -c 10.10.1.2 -u -b 100G
```

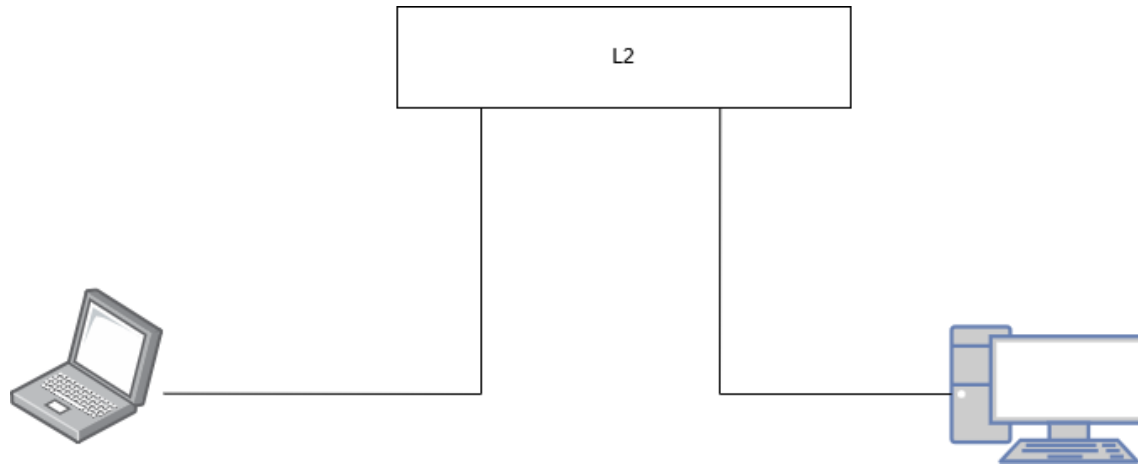
Test 2 - Result

```
user@user-OptiPlex-790:~/shivan$ iperf3 -c 10.10.1.2 -u -b 100G
Connecting to host 10.10.1.2, port 5201
[ 5] local 10.10.1.1 port 37567 connected to 10.10.1.2 port 5201
[ ID] Interval           Transfer     Bitrate      Total Datagrams
[ 5]  0.00-1.00   sec    215 MBytes  1.80 Gbits/sec  155386
[ 5]  1.00-2.00   sec    216 MBytes  1.81 Gbits/sec  156535
[ 5]  2.00-3.00   sec    226 MBytes  1.90 Gbits/sec  163611
[ 5]  3.00-4.00   sec    230 MBytes  1.93 Gbits/sec  166491
[ 5]  4.00-5.00   sec    232 MBytes  1.95 Gbits/sec  168037
[ 5]  5.00-6.00   sec    234 MBytes  1.96 Gbits/sec  169128
[ 5]  6.00-7.00   sec    234 MBytes  1.96 Gbits/sec  169189
[ 5]  7.00-8.00   sec    233 MBytes  1.95 Gbits/sec  168378
[ 5]  8.00-9.00   sec    233 MBytes  1.95 Gbits/sec  168535
[ 5]  9.00-10.00  sec    233 MBytes  1.95 Gbits/sec  168589
- - - - -
[ ID] Interval           Transfer     Bitrate      Jitter    Lost/Tot.  Datagrams
[ 5]  0.00-10.00  sec    2.23 GBytes  1.92 Gbits/sec  0.000 ms  0/1653879  (0%)  sender
[ 5]  0.00-10.00  sec    2.23 GBytes  1.92 Gbits/sec  0.003 ms  0/1653871  (0%)  receiver
iperf Done.
```

Test 2 result, PC1

b) PC to Laptop - through L2 switch

To contrast the results with a standard RJ45 connection, another setup was run



Topology for test

A PC and Laptop were connected to the L2 switch using RJ45 ethernet cable and ports. Static IPs were assigned, as mentioned previously.

Test 3 - TCP, L2 switch

- on Laptop:

```
iperf3 -s
```

- on PC1:

```
iperf3 -c 10.10.1.2
```

Test 3 - Result

```
user@user-OptiPlex-790:/etc/netplan$ iperf3 -c 10.10.1.2
Connecting to host 10.10.1.2, port 5201
[ 5] local 10.10.1.1 port 43418 connected to 10.10.1.2 port 5201
[ ID] Interval      Transfer    Bitrate      Retr  Cwnd
[ 5]  0.00-1.00    sec     96.9 MBytes  813 Mb/s     43   370 KBytes
[ 5]  1.00-2.00    sec    111 MBytes  935 Mb/s      0   370 KBytes
[ 5]  2.00-3.00    sec    111 MBytes  934 Mb/s      0   370 KBytes
[ 5]  3.00-4.00    sec    112 MBytes  936 Mb/s      0   370 KBytes
[ 5]  4.00-5.00    sec    111 MBytes  935 Mb/s      0   370 KBytes
[ 5]  5.00-6.00    sec    111 MBytes  933 Mb/s      0   370 KBytes
[ 5]  6.00-7.00    sec    111 MBytes  935 Mb/s      0   383 KBytes
[ 5]  7.00-8.00    sec    111 MBytes  935 Mb/s      0   383 KBytes
[ 5]  8.00-9.00    sec    111 MBytes  929 Mb/s      0   383 KBytes
[ 5]  9.00-10.00   sec    112 MBytes  936 Mb/s      0   383 KBytes
-----
[ ID] Interval      Transfer    Bitrate      Retr
[ 5]  0.00-10.00   sec    1.07 GBytes  922 Mb/s     43
[ 5]  0.00-10.04   sec    1.07 GBytes  917 Mb/s
sender
receiver
iperf Done.
```

Test 3 result, PC1

Test 4 - UDP, L2switch

- on Laptop:

```
iperf3 -s
```

- on PC1: UDP, by default, limits bitrate to 1Mbps. To test at higher speeds, we need to change the max bitrate. `-b` option is used to specify max bitrate.

```
iperf3 -c 10.10.1.2 -u -b 10G
```

Test 4 - Result

```
user@user-OptiPlex-790:/etc/netplan$ iperf3 -c 10.10.1.2 -u -b 10G
Connecting to host 10.10.1.2, port 5201
[ 5] local 10.10.1.1 port 48864 connected to 10.10.1.2 port 5201
[ ID] Interval           Transfer     Bitrate      Total Datagrams
[ 5] 0.00-1.00   sec    113 MBytes   949 Mbits/sec  81946
[ 5] 1.00-2.00   sec    113 MBytes   949 Mbits/sec  81937
[ 5] 2.00-3.00   sec    113 MBytes   949 Mbits/sec  81940
[ 5] 3.00-4.00   sec    113 MBytes   949 Mbits/sec  81945
[ 5] 4.00-5.00   sec    113 MBytes   949 Mbits/sec  81936
[ 5] 5.00-6.00   sec    113 MBytes   949 Mbits/sec  81946
[ 5] 6.00-7.00   sec    113 MBytes   949 Mbits/sec  81940
[ 5] 7.00-8.00   sec    113 MBytes   949 Mbits/sec  81943
[ 5] 8.00-9.00   sec    113 MBytes   949 Mbits/sec  81934
[ 5] 9.00-10.00  sec    113 MBytes   949 Mbits/sec  81944
- - - - -
[ ID] Interval           Transfer     Bitrate      Jitter        Lost/Total Datagrams
[ 5] 0.00-10.00  sec    1.11 GBytes   949 Mbits/sec  0.000 ms      0/819411 (0%)  sender
[ 5] 0.00-10.02  sec    1.10 GBytes   947 Mbits/sec  0.007 ms     41/819411 (0.005%) receiver

iperf Done.
```

Test 4 result, PC1